Thymeleaf CheatSheet for DAT108

Lars-Petter Helland, 18.okt 2026

Disclaimer: Fordi det ikke var enkelt å finne en tutorial som passet helt til DAT108-pensum, har jeg i stedet gitt instruksjoner via nettleser til Google sin KI om hva vi ønsker å ha med, og så fått generert opp aktuelle eksempler for dette.

Hva er Thymeleaf

Thymeleaf er en moderne, server-side Java-malmotor (template engine) som brukes til å generere og vise dynamisk innhold, som oftest i form av HTML-nettsider. Den brukes som regel i webapplikasjoner (spesielt sammen med Spring Boot) for å ta data fra Java-koden og flette det inn i HTML før siden sendes til brukerens nettleser.

Maven-avhengighet (pom.xml)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

I toppen av HTML-malene

```
<html xmlns:th="http://www.thymeleaf.org">
```

Hvordan Thymeleaf henter data

Kobling til Controlleren og Modellen:
Når du skriver ${x} i en HTML-mal, refererer dette direkte til en modell-attributt med nøkkelen (key) "x". Denne verdien må ha blitt lagt inn i modellen tidligere i den samme HTTP-forespørselen (requesten) av Java-kontrolleren din, typisk ved bruk av model.addAttribute("x", objekt).

Objektegenskaper:
Når du skriver ${x.y}, refererer du til egenskapen y inni objektet x. Det som skjer under panseret her er: Thymeleaf gjør *ikke* direkte oppslag mot private feltvariabler (som private String y;). I stedet vil Thymeleaf lete etter og kalle de **public** metodene på objektet. Den vil forsøke getY() (hvis person er av klasse-type), y() (hvis person er av record-type), eller isY() (hvis det er en boolean). Dersom ingen av disse public metodene eksisterer, vil applikasjonen feile med en feilmelding, selv om feltet finnes internt i Java-klassen.

Her er en kortliste over de aller viktigste Thymeleaf-bruksområdene.

Tekst, Verdier og Kommentarer

- **th:text** – Erstatter innholdet i et element med tekst. Sikker mot skript-injeksjon (HTML-escaped).

```
<p th:text="${bruker.navn}">Standardnavn</p>
```

- **th:utext** – Tolker og viser HTML-koder (Unescaped). Brukes om teksten din inneholder f.eks. <b> eller <i>.

```
<p th:utext="${bruker.biografi}">Tekst</p>
```

- **Thymeleaf-kommentarer** – Kommentarer som fjernes før siden sendes til nettleseren. De vises aldri i "Vis kildekode".

```
<!--/* Denne kommentaren og koden under slettes helt i nettleseren */-->
<!--/* <div th:text="${hemmeligData}"></div> */-->
```

Tekstkonkatenering (Sammenslåing)

- **Literal Substitution (|...|)** – Lar deg skrive ren tekst og variabler om hverandre uten å måtte bruke pluss-tegn (+) og en haug med apostrofer.

```
<!-- I stedet for: th:text="'Velkommen, ' + ${bruker.navn} + '!'" -->
<p th:text="|Velkommen, ${bruker.navn}!|">Standardtekst</p>
```

<th:block> (Rendring uten HTML-tags)

- **<th:block>** – Dette er en container-tag som kun eksisterer for Thymeleaf. Den kjører logikken du ber om, men selve tagen forsvinner helt fra den ferdige HTML-koden.

```
<!-- Lager kun <li>-elementene, ingen unødvendig HTML rundt -->
<th:block th:each="produkt : ${produkter}">
    <li><strong th:text="${produkt.navn}">Vare</strong></li>
</th:block>
```

Løkker og Lister (Iterasjon)

- **th:each** – Går gjennom en samling. Du definerer en variabel for elementet (produkt), og kan valgfritt hente en status-variabel (status) for indekser.

```
<ul>
  <li th:each="produkt, status : ${produkter}">
    <span th:text="${status.count}">1</span>.
    <span th:text="${produkt.navn}">Melk</span>
  </li>
</ul>
```

Betingelser (Vis/Skjul)

- **th:if** – Viser elementet kun dersom uttrykket er sant.

```
<div th:if="${bruker.erInnlogget}">Velkommen tilbake!</div>
```

- **th:unless** – Det motsatte av th:if. Viser elementet kun dersom uttrykket er usant.

```
<div th:unless="${bruker.erInnlogget}">Vennligst logg inn.</div>
```

- **th:switch / th:case** – Velger mellom flere alternativer. Bruk * for standardvalget (default).

```
<div th:switch="${bruker.rolle}">
  <p th:case="'ADMIN'">Du er sjefen.</p>
  <p th:case="'BRUKER'">Du er vanlig bruker.</p>
  <p th:case="*">Ukjent rolle.</p>
</div>
```

Nyttige Expression Language -operatorer

- **Elvis-operatoren (?:)** – Gir deg en standardverdi (fallback) dersom verdien før spørsmålstegnet er null eller tom.

```
<!-- Hvis bruker.kallenavn er null, vises 'Gjest' i stedet -->
<p th:text="${bruker.kallenavn ?: 'Gjest'}">Visningsnavn</p>
```

- **Betinget-operatoren (if ? then : else)** – En kjapp en-linjers if/else.

```
<p th:text="${bruker.alder >= 18 ? 'Myndig' : 'Mindreårig'}"></p>
```

Unngå NullPointerException (Safe Navigation)

- **Safe Navigation-operator (?.)** – Stopper hvis objektet foran er null, i stedet for å krasje.

```
<!-- Hvis adresse er null, hopper den over 'gate' og skriver ingenting -->
<p th:text="${bruker.adresse?.gate}">Gatenavn</p>
```

- **Dobbeltsjekk med th:if** – Sjekk at objektet faktisk eksisterer før du prøver å bruke det.

```
<div th:if="${bruker != null}">
    <p th:text="${bruker.navn}">Navn</p>
</div>
```

Innebygde Objekter (Kontekst)

- **param** – Henter ut URL-parametere (Query parameters).

```
<!-- Hvis URL-en er /side?sok=kaffe, vil dette skrive ut 'kaffe' -->
<p th:text="${param.sok}">Søketekst</p>
```

- **session** – Henter verdier som ligger lagret i brukerens HTTP-session.

```
<p th:text="${session.handlekurv.antallVarer}">0</p>
```

- **#strings, #lists, #numbers** – Innebygde hjelpeverktøy for å manipulere data (f.eks. sjekke lengde på lister eller formatere tall).

```
<!-- Sjekker om en liste er tom uten å krasje -->
<div th:if="${#lists.isEmpty(produkter)}">Listen er tom</div>
```

Ekstra ...

Lenker og Ressurser

- **th:href** – Brukes på <a>-lenker og <link>-elementer. Man bruker @{...} for URL-er.

```
<a th:href="@{/profil/rediger}">Rediger profil</a>
<!-- Med dynamisk ID: -->
<a th:href="@{/produkt/detaljer(id=${produkt.id})}">Se produkt</a>
```

- **th:src** – Brukes for bilder, skript og andre eksterne ressurser.

```
<img th:src="@{/images/logo.png}" alt="Logo" />
```

Skjemaer (Form Handling)

- **th:action** – Setter målet for hvor skjemaet skal sendes.

```
<form th:action="@{/lagre-bruker}" method="post">
```

Gjenbruk av kode (Layout/Templates)

- **th:fragment** – Definerer en bit med HTML som kan gjenbrukes andre steder.

```
<!-- I filen felles.html -->
<div th:fragment="meny">
    <nav>...</nav>
</div>
```

- **th:insert / th:replace** – Henter inn et fragment fra en annen fil. replace erstatter vertstaggen fullstendig, mens insert dytter fragmentet inn i vertstaggen.

```
<!-- Erstatter denne div-en med meny-fragmentet fra felles.html -->
<div th:replace="~{felles :: meny}"></div>
```